

How My Tests Work

demoblaze automation — a plain-English walkthrough of both test suites

Repository: BlazaMeterWebsite-Automation-Seleniu-with-Python- | Branch: POM-Pytest

1. The big picture

You now have **two separate sets of tests** in one project. They test the same website in two completely different ways.

Suite	What it does	How it feels
UI tests (Selenium)	Opens a real Firefox window and clicks through the website like a human being would.	Slow (about 96 seconds), realistic, breaks easily if the page design changes.
API tests (requests)	Talks straight to the server behind the website. No browser at all.	Fast (about 16 seconds for 14 tests), stable, but never sees the actual page.

Think of it like testing a restaurant. The **UI test** walks in, sits at a table, reads the menu and orders out loud. The **API test** phones the kitchen directly and asks “do you have the Samsung Galaxy S6, and how much is it?” Both are useful. They catch different problems.

Important: Selenium *cannot* do API testing. Selenium's only job is to control a browser. To test an API you need a different tool — here it is a Python library called `requests`. The two simply live in the same project and run with the same command.

2. The UI test, step by step

When you run `pytest -m ui`, exactly one test runs: `test_purchase_flow`. It performs the whole “buy something” journey and **checks the result of every step**. Below, each step shows the action and then, in green, what is verified afterwards.

1. **Firefox opens** and goes to demoblaze.com. (done by the `Browser` fixture in `conftest.py`)
2. **Clicks “Sign up”**, types `Shahrukh` / `12345`, submits.
3. **Reads the popup and clicks OK.**
✓ the message must be exactly “This user already exist.” — if it ever says “Sign up successful”, the account has vanished.
4. **Closes** the sign-up box.
5. **Clicks “Log in”** in the top menu.
6. **Types the username.**
7. **Types the password.**

8. **Clicks the “Log in” button.**

- ✓ the navbar must read “Welcome Shahrukh”
- ✓ the “Log out” link must be visible

9. **Clicks the product** “Samsung galaxy s6”.

- ✓ the product page title must be “Samsung galaxy s6”
- ✓ the price must be 360

10. **Clicks “Add to cart”.**

11. **Accepts the popup.**

- ✓ the message must be exactly “Product added.”

12. **Clicks “Cart”.**

- ✓ the cart must contain exactly one row: Samsung galaxy s6
- ✓ the cart total must be 360

13. **Clicks “Place Order”.**

- ✓ the order form must actually open
- ✓ its total must match the cart total

14. **Types the name** shahrukh .

- ✓ the text must really be in the field

15. **Types the card number** 4111 1111 1111 1111 .

- ✓ the number must really be in the field

16. **Clicks “Purchase”.**

- ✓ the confirmation must say “Thank you for your purchase!”
- ✓ it must show “Amount: 360 USD”
- ✓ it must show the card number that was typed

17. **Clicks “OK”.**

- ✓ the browser must be back on the home page
- ✓ **the cart must now be empty**

Seventeen checks in total. Three of them would catch a genuine money bug: the cart total matching the product price, the order form total matching the cart, and the cart being emptied after a completed purchase. A shop that charges the wrong amount or forgets to clear the basket now fails loudly instead of passing quietly.

How the code is organised

Those 17 steps are not written in one long file. They are split up, which is called the **Page Object Model**:

test_purchase_flow.py	the test	-> "run the purchase journey"
Test/Action.py	the recipe	-> the 17 steps in order, and all 17 checks
Pages/*.py	the steps	-> each file knows one part of the site

The rule of thumb: **page objects do things and read things; the flow decides whether the answer is right.** So `Pages/Test_Cart.py` knows how to fetch the cart total, and `Test/Action.py` is where it is compared against 360. That way `Test/Action.py` reads like a description of what the shop is supposed to do.

The other benefit: if demoblaze renames its “Add to cart” button, you change **one line in one file** instead of hunting through every test.

3. The API tests, step by step

When you run `pytest -m api`, 14 tests run in about 16 seconds and no browser opens. Here is what happens, in order.

Step 1 — Log in once

Before any test runs, the suite sends the username and password to `https://api.demoblaze.com/login` and gets back a **token** — a short code that proves “I am Shahrukh”. Every later test reuses that same token instead of logging in again.

Step 2 — Check the product catalogue (6 tests)

- Ask for **all products** and check every one has an id, title, price, category, image and description — and that the price is actually a number, not text.
- Ask for **only phones**, then **only notebooks**, then **only monitors**, and check nothing from another category sneaks into the answer.
- Ask for a **category that does not exist** and check the server politely returns an empty list instead of crashing.
- Ask for **one specific product** (id 1) and check it really is the Samsung galaxy s6 at \$360.

Step 3 — Check logging in (4 tests)

- Log in with the **right password** → a token comes back.
- Log in with the **wrong password** → the server must say exactly “Wrong password.”
- Log in as a **user who does not exist** → must say exactly “User does not exist.”
- Sign up with a username that is **already taken** → must say “This user already exist.”

That last one is deliberate. demoblaze is a public website shared by thousands of learners, and it has no way to delete an account. If the tests created a new user on every run, we would be leaving rubbish on someone else's server forever. So we test sign-up using the account that *already* exists — real coverage, nothing created.

Step 4 — Check the shopping cart (4 tests)

- **Add an item** to the cart, then look at the cart and confirm it is there.
- **Use a fake token** and confirm the server refuses with “Bad parameter, token malformed.”
- **Delete the item** and confirm the cart is empty again.

Every cart test cleans up after itself. Even if a test fails halfway, the item is still removed — so your cart never fills up with leftovers from old runs.

4. Two surprises we found in demoblaze

Surprise 1: the server says “200 OK” even when things go wrong

Normally a website answers with a number: **200** means “fine”, **404** means “not found”, **401** means “you’re not allowed”. demoblaze answers **200 for absolutely everything** — even for a wrong password.

```
Wrong password      -> 200 OK   {"errorMessage": "Wrong password."}
User does not exist -> 200 OK   {"errorMessage": "User does not exist."}
Complete nonsense   -> 200 OK   {"Items": []}
```

This matters a lot. Most tutorials write `assert response.status_code == 200` and think they have tested something. On this site that line **passes even when the request failed**. So our tests ignore the number and read the actual message instead. That logic lives in one file, `API/asserts.py`, so it is written once rather than fourteen times.

Surprise 2: the delete button does not delete

demoblaze has an endpoint called `/deletecart`. It replies “**Item deleted.**” — and the item is still sitting in the cart. We checked repeatedly; it is not a delay. A different endpoint, `/deleteitem`, is the one that actually works.

```
/deletecart -> "Item deleted."   then check the cart -> item is STILL THERE
/deleteitem  -> "Item deleted."   then check the cart -> cart is empty
```

You found a genuine bug in the website you are testing. That is exactly what testing is for. The test `test_deletecart_removes_the_item` is marked `xfail`, which means “we expect this to fail”. It shows as **XFAIL** in the report instead of turning your run red. And if demoblaze ever fixes it, the test will loudly tell you so you can remove the marker.

5. How do you know a test actually works?

A test that passes tells you very little on its own. A test with a mistake in it — checking nothing, or checking something that is always true — also passes, every single time, forever. It looks like safety and provides none.

The way to find out is to **break something on purpose and make sure the test notices**. When the 17 checks were added, the expected price was temporarily changed from 360 to 999 and the suite was run again:

```
assert price == PRICE, f"expected ${PRICE}, the page says ${price}"
E   AssertionError: expected $999, the page says $360

1 failed in 55.46s
```

It failed, at the right line, with a message that explains itself. Then the 999 was put back and the suite passed again. Now the green result means something.

Habit worth keeping: every time you write a new check, break it once and watch it fail before you trust it passing. It takes a minute and it is the difference between a test suite and decoration.

6. What every file does

File	Its job, in one line
<code>conftest.py</code>	Opens and closes Firefox; saves a screenshot when a UI test fails.
<code>pytest.ini</code>	Settings: which files are tests, and the <code>api</code> / <code>ui</code> labels.
<code>test_purchase_flow.py</code>	The one UI test.
<code>Test/Action.py</code>	The 17 UI steps in order — and all 17 checks.
<code>Pages/*.py</code>	One file per part of the website: clicks it, and reads it back.
<code>API/config.py</code>	The website address, username and password.
<code>API/client.py</code>	Knows how to talk to the server — the API version of the Pages files.
<code>API/asserts.py</code>	Decides whether an answer counts as success or failure.
<code>API/conftest.py</code>	Logs in once; adds and removes cart items around each test.
<code>API/test_*.py</code>	The 14 API tests themselves.
<code>reports/</code>	HTML reports you can open in a browser.
<code>screenshots/</code>	Pictures saved automatically when a UI test fails.

7. How to run everything

```
pip install -r requirements.txt      install the tools (once)

pytest -m api                       just the API tests    (fast, no browser)
pytest -m ui                        just the UI test      (opens Firefox)
pytest                             both

pytest -m api --html=reports/api_report.html --self-contained-html
...and save a report you can open
```

The `-m` means “only run tests with this label”. The labels come from the `@pytest.mark.api` and `@pytest.mark.ui` lines in the test files.

8. Reading the result

```
13 passed, 1 xfailed, 1 deselected in 15.75s
```

Word	What it means
passed	The test checked something and it was correct.
failed	The test checked something and it was wrong. Read the message underneath.
xfailed	Failed <i>on purpose</i> — a known bug we already know about. Not your problem.
xpassed	A known bug got fixed! Go and remove the <code>xfail</code> marker.
deselected	Skipped because of the label — e.g. the UI test when you ran <code>-m api</code> .
error	The test never even started — usually the setup broke (browser, network).

9. Honest list of things to improve next

1. **Replace `time.sleep` with proper waits.** The UI flow contains about **64 seconds** of fixed sleeping, which is most of its 96-second runtime. `WebDriverWait` waits only as long as it actually needs to — usually a fraction of a second — and is *more* reliable, not less, because it also waits longer on a slow day. This is the biggest win available.
2. **Rename the files in `Pages/`.** They are called `Test_Login.py` with `class TestLogin`, so pytest used to mistake all 17 page-object methods for real tests and run them in the wrong order. It is currently patched with a setting in `pytest.ini`; renaming them to `login_page.py` / `class LoginPage` is the proper fix.
3. **Use the API to speed up the UI test.** Log in through the API and hand the token to the browser, skipping UI steps 2–8 entirely. That is the normal next step in a real framework, and it is now possible because the API layer exists.

4. **Split the one big UI test into a few smaller ones.** Today a failure at step 4 means steps 5–17 never run, so you learn about one problem per run. Separate tests for login, for the cart and for checkout would each report independently.
5. **Run the API suite automatically.** A GitHub Actions workflow could run `pytest -m api` on every push — it needs no browser, so it is easy to set up and takes under a minute.